

### Deterministic Finite Automaton (DFA)

A **DFA** is a mathematical model used in *Theory of Computation* to recognize **regular languages**. It is called *deterministic* because its behavior is completely predictable: for every state and every input symbol, there is exactly **one possible transition**.

#### Characteristics of DFA:

- For each state, there is one transition for each input symbol.
- It does not allow  $\lambda$ -transitions.
- The computation follows a single path from start to end.
- Every state must have transitions for all symbols.

#### Advantages:

- Simple and easy
- Clear and predictable behavior
- Only one computation path

#### Disadvantages of DFA

- require a **large number of states**
- harder to design compared to NFA

#### Example:

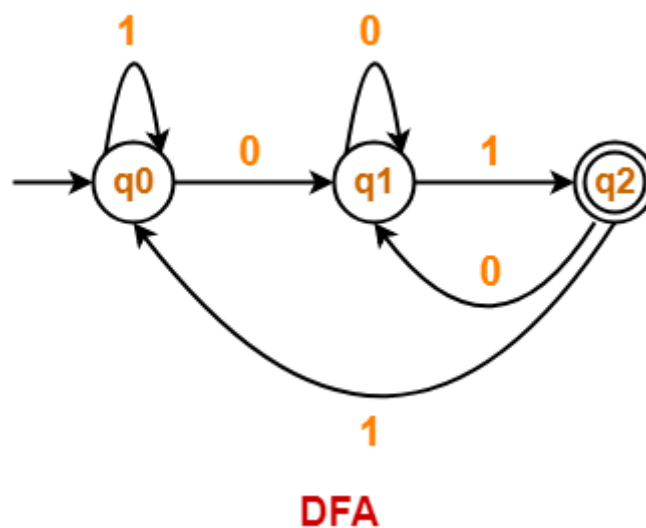


Figure 1: DFA Example

---

## Non-Deterministic Finite Automaton (NFA)

NFA is more flexible than DFA. It allows multiple possible transitions for the same input symbol from the same state and may include  $\lambda$ -transitions.

### Characteristics of NFA:

- A state may have more than one path for the same input
- It may include  $\lambda$ -paths
- It is not required to define a path for every symbol
- The computation may follow multiple paths

### Advantages:

- Easier to design.
- More flexible.
- Requires fewer states.

### Disadvantages of NFA

- Harder to implement directly
- Requires tracking multiple computation paths

### Example:

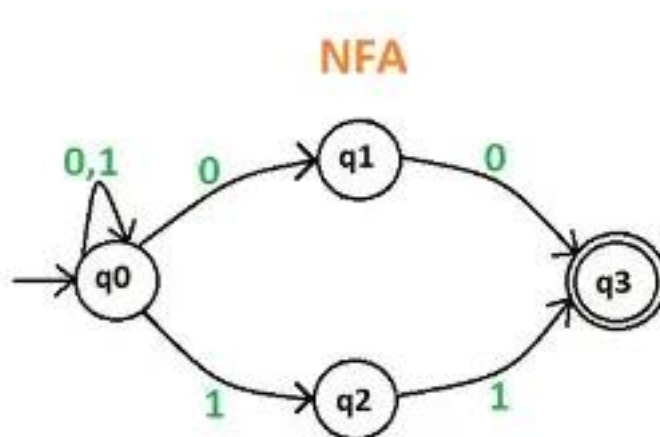


Figure 2:NFA Example